INF1103: Programming Fundamentals with DevOps

Week 11: Design Patterns, Error Handling and CI | CD

Content

- A few points from last week
- Software Design Patterns
- Error Handling
- CI | CD Pipeline

A few points from last week



Speak()

Sleep()

Eat()



FetchStick()

Speak()

Override : Woof



ClimbTree()

Speak()

Override : Meow

Polymorphsim

We only care it is animal
animal.speak(), method depends on which animal is stored
If it is a dog, override parent a say woof, if it is a cat override parent and say meow

```
animals = [Dog(), Cat()]

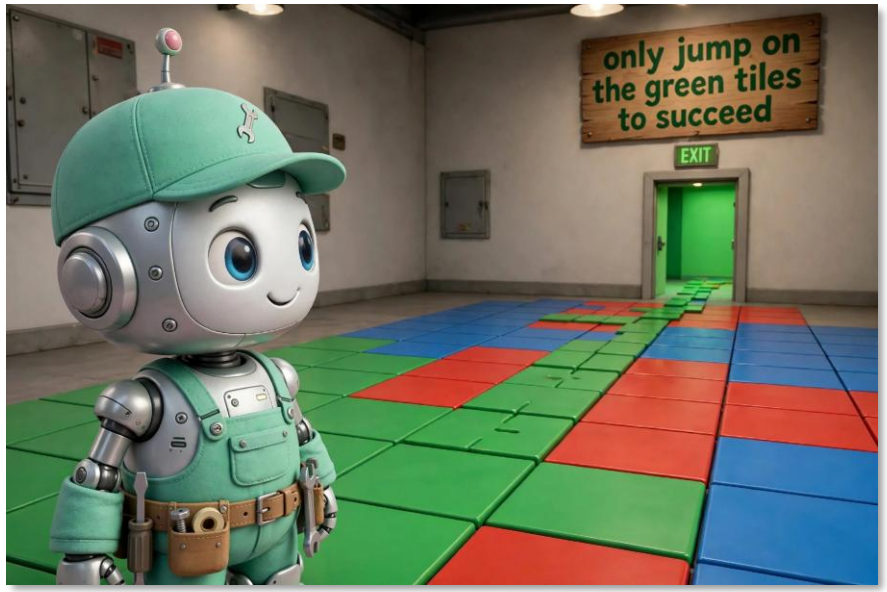
for animal in animals:
    print(animal.speak())
```

Software Design Patterns

- In life:
  - What is a pattern?
  - What do they teach us?
  - Why are they useful?
  - What happens if we don’t share patterns?
- What is a design?
- So, a design pattern is?

Software Design Patterns

- Software design patterns are **solutions** to **common problems** that have been tried and tested to a point that they become a pattern
- They form based on the principles of:
  - A recurring problem solved in a number of situations in a similar way
  - The details or context may change but the design challenge is the same
  - Agreement is made on a common ‘good’ standard approach
  - A design pattern is born from this approach



Software Design Patterns

- **WHAT** are they:
  - A general, **reusable solution** to a commonly occurring software design problem
  - Describes the structure and collaboration of classes and objects
  - It acts as a **solution template** not the finalized code
  - Patterns act as a **shared vocabula**ry for developers
  - Help **articulate and discuss design choices** and trade offs.





Software Design Patterns

- **WHAT** they are NOT:
  - Not a library you import, they advise on how to write your code
  - Not a framework that writes the application for you, they inform your design and implementation strategy
  - Not a rule that must be followed, they are a **design decision**
  - Not always the best path but potentially a good one
  - Not to be blindly implemented, can result in code complexity to a potentially simple problem

Hint: Use the simplest design that solves the current problem best.

Software Design Patterns

- **WHY** should we use them:
  - Tried and tested designs **prevent us reinventing existing solutions**
  - Often **separate responsibilities** well so **improve maintainability**
  - **Enhance extension** when change are required
  - Other developers will be **familiar** with them so improve code **understandability**
  - Can help **reduce** some forms of **tight coupling** of code

Why? Because Software Developers always care about these things!

Software Design Patterns

- GoF : Refers to the Gang of Four based on the 1994 book Design Patterns: Elements of Reusable Object-Oriented Software written by four authors
  - Erich Gamma
  - Richard Helm
  - Ralph Johnson
  - John Vlissides
- They became known as the “Gang of Four” and outlined 23 design patterns which are still heavily used today.

Software Design Patterns

- There are 23 Software Design Patterns from GoF which are categeorised into 3 different functional areas:

Creational	Structural	Behavioural
1. Singleton	6. Adapter	13. Template Method
2. Factory	7. Composite	14. Mediator
3. Abstract Factory	8. Proxy	15. Chain of Responsibility
4. Builder	9. Fly Weight	16. Observer
5. Prototype	10. Façade	17. Strategy
	11. Bridge	18. Command
	12. Decorator	19. State
		20. Visitor
		21. Iterator
		22. Interpreter
		23. Memento

Software Design Patterns

- Let’s learn in depth all 23 patterns!

- Kidding! We will just cover 2 to give an understanding of what they are all about
  - Strategy (17th Pattern in the list)
  - Factory (2nd Pattern in the list)

Strategy Pattern

- **Problem:**
  - One tasks can be performed in multiple different ways
  - The most obvious solution is to put every option into one large if/elif block
  - As the options to perform a task grow, it gets harder to change and test

Why is the above problem bad?

- **Strategy:**
  - Move each option (behaviour) into its own class
  - The context uses a strategy through a shared method

Idea: How to add a new context without changing the context class

Strategy Pattern

- ***Without a good pattern:***
  - The class contains every possible way to perform a task
  - Every time a new option is added we need to edit the method
  - The class breaks **O**pen **C**lose **P**rinciple (OCP)

Example: export method that exports csv and json

```
def export(self, format, data):
        if format == “csv”:
                print (“Exporting CSV”)
        elif format ==“json”:
                print (“Exporting JSON”)
        else:
                raise ValueError(“Unsupported format”)
```

Client wants xml export, export method must change
Class doesn’t follow OCP

```
def export(self, format, data):
        if format == “csv”:
                print (“Exporting CSV”)
        elif format ==“json”:
                print (“Exporting JSON”)
        elif format == “xml”:
                print (“Exporting XML”)
        else:
                raise ValueError(“Unsupported format”)
```

Strategy Pattern

- ***With the strategy pattern:***
  - Each strategy provides the same operation: export (data)
  - The report uses the elected strategy but doesn’t need to know about its internal details (how it does the export)
  - Every time a new strategy is required, create a new class rather than edit the export method.

Strategy Pattern

```
class ExportFormat(ABC):
@abstractmethod
        def export(self, data):
                pass
```

Create abstract class to ensure structure (children must implement export)

```
class CSVExport(ExportFormat):
        def export(self, data):
                print("Exporting data as CSV")
```

```
class JSONExport(ExportFormat):
        def export(self, data):
                print("Exporting data as JSON")
```

Inherits ExportFormat and implements own version of export

```
class Report:
    def __init__(self, strategy):
        self.strategy = strategy

    def export(self, data):
        self.strategy.export(data)
```

Report class takes strategy as parameter
Doesn't matter if csv or json exports are in strategy, as long as they implement export

What object we pass determines which export is called!
In this case JSONExport object

External Code:

```
report = Report(JSONExport())
report.export(records)
```

Strategy Pattern

- Client wants to add XML Export?
  - No problem, create an XML Export class:

```
class XMLExport(ExportFormat):
        def export(self, data):
                print("Exporting data as XML")
```

  - Note the Report Class doesn't change (OCP achieved):

```
class Report:
    def __init__(self, strategy):
        self.strategy = strategy

    def export(self, data):
        self.strategy.export(data)
```

```
External Code:

report = Report(XMLExport())
report.export(records)
```

Strategy Pattern

- **Reflection on the pattern:**
  - The behaviour is separated from the class that uses it
  - Report class is only concerned with strategy behaviour, not on how CSV or JSON or XLM is implemented (in fact it doesn't know about these classes at all)
  - We can change the strategy without changing any code in Report class
  - We can add strategies without changing code in Report Class
  - Report is now successfully open for extension but closed for modification

Factory Pattern

- **Problem:**
  - Object creation is becoming complicated or repeated
  - The client code should not need to know much about object creation

Why is the above problem bad?

- **Strategy:**
  - Provide a central place to decide which object to create
  - If a client asks for an object, this ‘central place’ handles the creation decisions
  - Let’s call this central place ‘a factory’

Factory Pattern

- **Without a good pattern:**
  - Creation decisions are repeated in different parts of the program
  - Client code becomes coupled to concrete classes
  - Changing how an object is created may required changes in code across your application

Some Client Class

```
if record_type == “student”:
        record = StudentRecord(name, score)
elif record_Type == “staff”:
        record = StaffRecord(name, department)
else:
        raise ValueError(“Unknown record type”)
```

Client does not have ‘create object’ as a responsibility, but it is creating classes (SRP issue)

New record types means this client has to change (OCP issue)

Client must know about all record type classes (tight coupling)

Factory Pattern

- ***With the Factory Pattern:***
  - The factory owns the creation decision
  - The rest of the program asks for the type of record it needs
  - If creation rules change, we have one place to update

Factory Pattern

```
class RecordFactory:
@staticmethod
def create(record_type, name, value)
        if record_type == “student”:
                return StudentRecord(name, value)
        elif record_type == “staff”:
                return StaffRecord(name, value)
        else
                raise ValueError(“Unknown Record Type”)
```

Single responsibility of creating Records (SRP solved)

RecordFactory needs to know of all record types, so sensible tight coupling

Class still needs to change for new records (moved OCP issue to here but ok for now!)

```
Some Client Class
record = RecordFactory.create("student", "Asha", 85)
```

Factory Pattern

- Reflection on implementation
  - Factory Pattern means only the **factory** is **responsible** for **object creations**
  - **Decouples code**, clients don't need to know about object types, only factory does, clients need only know about factory
  - Client no longer has 'object creation' as responsibility
  - **OCP** removed from client
  - **OCP remains in factory**, there are ways around this but for this module and to avoid over complicating implementation, this is acceptable

Note:(remember SOLID are principles not fixed rules, as long as you can explain your hopefully justified design decision)

Design Pattern Take Aways

- Patterns describe proven ways to structure collaborating objects
- Strategy separates interchangeable behaviour
- Factory centralises object creation
- Both can reduce coupling and support maintainability
- Recognise the problem first; choose a pattern only if it helps

Error Handling

- What is happening here?
  - Why does the father look worried?



Error Handling

- Often referred to as TRY / CATCH
- This means, we try some code and we catch an issue if it occurs
- The issue is called an ‘exception’ so in python we call it:
  - Try / Except



Error Handling

- In any program **unexpected things** beyond our control can happen such as:
  - Missing files or Fields
  - Invalid Inputs
  - Unavailable API's
  - Network timeout
  - Logic issues such as divide by zero
- As developers we try to **predict** possible issues and ensure our application **fails 'gracefully'** (no runtime errors!).
  - A graceful failure means we don't prevent every possible error, but we catch the problem as it occurs, provide an error message and ensure our application continues to run

Error Handling

- Some terminology to start with:
- **Error :** a problem that prevents a program running as intended
- **Exception :** An event raised during execution that interrupts the normal flow
- If an **exception** is not **handled** by your code, your program will likely terminate (not fail gracefully).

Error Handling

- There are two types of error checks
  - **Validation :** Check if data meets expected rules before processing
  - **Exception Handling :** response to an operation raising an exception
- Validation should be used on expected user / business rules
- Exceptions for failures are used when an operation is attempted that may result in failure if all conditions for that operation are not met
- Validation and Exception handling work together in your code and are not replacements for each other

Error Handling

- Validation is usually done via quick logic checks before business logic is run (if else statements)
- Exception Handling is done when the developer expects an exception ‘could’ happen
- As a developer you should know when to expect an exception and handle the exception you expect to occur
- When an exception happens we say an exception was **thrown**
- For exception handling we can use **try / catch** for
  - ValueError : Conversion or value is not valid
  - FileNotFoundError : A file or path does not exist
  - PermissionError : Access to a file is denied
  - ZeroDivisionError : Division by Zero
  - OSError : a broader operating-system / file-related failure

Error Handling

- Try / Except (Try / Catch)

```
try:
        age = int (input("Enter your age: "))
except ValueError:
        print("Please enter a whole number.")
Print ("Program Continues...")
```

We are going to ***try*** this code, but we know users may enter incorrect **values**

We expect a value error so we account for this exception

We **catch** the error and print a message to the user

The code continues to run with the error caught rather than terminate the code

Error Handling

- Try / Except / Else / Finally

```
try:
        data = file.read()
except FileNotFoundError:
        print("The records file was not found.")
else:
        print("File loaded successfully.")
finally:
        print("Load attempt finished.")
```

We are going to ***try*** this code, but we know if the file is moved or not created it may not exist

We account for this exception and print a message that the file hasn't been found

IF the file IS found, happy days! We print a message to say the file has been loaded

Finally is code that runs after a try regardless of if an exception was thrown or not

Error Handling

- Good practice for error handling:
  - Catch exceptions at a level where you can make useful decisions
  - Keep error messages simple and understandable to users
  - Keep technical details only for debugging and logs
  - Be specific with your except, no broad (don’t try to do too much or use it too often where it may not be needed)
  - Do not continue with data that is incomplete due to an except, get the user to try again until you have what you need
  - Clean up resources when necessary

CI / CD Pipeline

- **CI/CD** stands for **Continuous Integration** and **Continuous Delivery (or Continuous Deployment)**
- It is a software development methodology that **automates building, testing, and releasing code**

CI / CD Pipeline

- **CI:**
  - Developers merge code changes frequently into a shared central repository.
  - Each **merge triggers an automated build** and **a suite of tests**
  - This allows bugs and integration conflicts to be identified early, closer to the change that introduced them
  - CI gives the team fast feedback, but doesn’t guarantee bug-free software

CI / CD Pipeline

- **CD:**
  - Validated code from the CI process is **automatically packaged and prepared for release.**
  - It keeps the application in a **constantly deployable state**
  - pushing to production typically requires a manual approval step.
  - For this module, it is important to know what CD is, but we will focus a little more on CI

CI / CD Pipeline

- A simple CI pipeline may look as follows:
  - Developer commits and pushes code to GitHub
  - GitHub Actions starts a workflow
  - Set up the required Python environment
  - Install project dependencies
  - Run automated checks / tests
  - Report success failure to the team

CI / CD Pipeline

- A workflow is triggered by a push or pull request
- The runner checks out the repository
  - A **runner** is the physical or virtual machine—or the background agent process—that actually executes the jobs defined in your pipeline
- Python and required dependencies are installed
- Tests run automatically
- A failed test make the workflow fail
- The team investigate failures before merging / releasing

CI / CD Pipeline

```
name: Python checks

on: [push, pull_request]

jobs:
 test:
  runs-on: ubuntu-latest
  steps:
   - uses: actions/checkout@v4
   - uses: actions/setup-python@v5
     with:
       python-version: "3.12"
   - run: pip install -r requirements.txt
   - run: python -m unittest discover
```

- Readable name for your workflow
- Defines the event that triggers the workflow
- Describes the work that will be performed
- Selects the machine (runner) that will perform the work
- List the actions or commands in order

If the workflow completes, a green tick is given, this doesn't mean software is perfect it just means the tests passed

CI / CD Pipeline

- CI checks more than just ‘does it run’:
  - Unit tests: do individual functions / classes behave as expected?
  - Integration tests: do components work together?
  - Static checks / linting: are there obvious style or code-quality issues?
  - Build checks: can the application or package be built?
  - Start small — a few meaningful tests are better than no tests

Thanks!!!

